

Web Page Tampering Detection Based on Dynamic Temporal Graph Pre-Training

Yijia Xu , Qiang Zhang , Kaiyang Wang , Zhonglin Liu , Cheng Huang , *Senior Member, IEEE*,
and Yong Fang 

Abstract—Web page tampering detection is crucial in web threat perception. Current methods rely on monitoring historical changes of web pages to identify anomalies. These approaches often struggle to effectively distinguish between tampering and benign changes, especially in the presence of numerous dynamic pages. Furthermore, the increasing complexity of website structures places more resource demands on tampering monitoring and makes some malicious alterations more covert and challenging to detect. We propose a web page tampering detection based on pretraining with dynamic temporal graphs. The core of the method involves constructing a website temporal graph model based on evolutionary information, and enhances the graph feature perturbations to expose concealed tampering behaviors. Specifically, the framework’s autoencoder is composed of enhanced DySAT, enabling it to handle dynamic data. We introduce DySAT, bolstered with GATv2, to capture dynamic attention. Additionally, we design a temporal masking mechanism and prediction error to improve the effectiveness of generative self-supervised learning in temporal graph pretraining. Experimental results on Webpage Tampering Dataset (WPT-Dataset) demonstrate that our method outperforms other comparative approaches in terms of both detection efficacy and stability. Furthermore, the research findings on the anomaly detector and model performance provide direction for the practical application of our method.

Index Terms—Web security, pre-training, dynamic graph, temporal data, anomaly detection.

I. INTRODUCTION

ATTACKERS have various means to compromise web pages, including social engineering, network attacks, and virus implantation. With unknown attacks emerging as a top threat, a security system can offer only partial protection for Web applications [1]. Web page tampering attackers are often individuals or organizations with strong Web attack capabilities who attempt to alter web page content for illicit purposes, such as spreading malware, conducting phishing attacks, disseminating false information, damaging brand reputation, and stealing sensitive information. As the defenders, website administrators must introduce appropriate defense mechanisms to monitor changes to web pages, assess tampering risks, respond swiftly,

and restore the site. When web page tampering occurs without detection, the personal privacy information of website users is at significant risk of exposure, and website operators may suffer reputational and possibly financial losses. Consequently, timely detection of web page tampering becomes crucial. Its purpose is to ensure the integrity and authenticity of web pages to prevent them from causing bad influence under the control of criminals [2].

Web page tampering detection is commonly integrated as a functional module within the Web Application Firewall (WAF). Four prevalent technologies serve real-world network environments: time polling, kernel embedding, event triggering, and file filtering [3]. Each method offers distinct advantages but also suffers from inherent limitations. Time polling, which periodically compares snapshots of web content, may miss short-lived tampering events and incur high resource costs. Kernel embedding monitors changes at the system or file level but is often constrained by operating system compatibility and deployment complexity. Event triggering provides real-time alerts based on specific system or application-level events, yet it can struggle to distinguish between benign and malicious behavior in dynamic web contexts. File filtering inspects content for unauthorized modifications, but may be bypassed by obfuscated or subtle tampering techniques. As a result, practical deployments often combine multiple techniques to improve coverage, though challenges persist. Given the limitations and vulnerabilities of individual methods, integrating several approaches into production is common to achieve comprehensive web page tampering detection.

The evolution of hacking techniques has heightened the challenge of detecting web page tampering, leading to various technical hurdles in existing detection methods. Here’s a summary of these key challenges: (a) Dynamic nature of web pages. Modern websites, particularly those emphasizing interaction and user experience, automate the updating of web page content in response to user interactions, data inputs, or other real-time events [4]. Technologies like JavaScript, AJAX, or server-side scripting languages commonly achieve this dynamic updating of web pages, enabling seamless real-time updates and sophisticated data handling. However, distinguishing between malicious tampering and routine changes becomes difficult for highly dynamic web pages. (b) Complex site structure. Extensive websites comprising numerous pages and external links present a daunting task for tamper detection. As websites become more complex, the factors to consider in web page tampering detection become

Received 29 April 2024; revised 11 October 2025; accepted 15 October 2025. Date of publication 20 October 2025; date of current version 12 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U20B2045. (Corresponding authors: Qiang Zhang; Yong Fang.)

The authors are with the School of Cyber Science and Engineering, Sichuan University, Chengdu 610065, China (e-mail: zhangqianges@stu.scu.edu.cn; yfang@scu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3623257

more uncertain [5]. Unfortunately, greater website complexity often leads to diminished efficiency in detecting tampering. (c) Concealed tampering. While some web page tampering seeks to create a conspicuous impact, others covertly embed hidden links on the page to achieve long-term gains, such as improved Search Engine Optimization (SEO) rankings [6]. Such tampering alters minimal content, frequently eluding detection under the guise of web page dynamics. Addressing these technical bottlenecks is pivotal for enhancing web page tampering detection and safeguarding the integrity and trustworthiness of online content.

To overcome these technical challenges, exploring novel solutions for web page tampering detection is essential. The core lies in addressing the dynamic changes, complex structures, and covert alterations of websites. Contemplating the following questions is vital to breaking through:

- *Q1*: How to model comprehensively considering the website structure and page dynamics?
- *Q2*: How to construct feature representations based on the temporal changes of web pages?
- *Q3*: How to amplify anomalous perturbations to uncover covert web page tampering?

Regarding *Q1*, we treat web pages as nodes and their linking relationships as edges, constructing a graph structure to model complex websites. To capture the dynamic changes in web pages, we establish a timeline and record graph snapshots at different timestamps, creating a dynamic temporal graph. For *Q2*, we extract three types of features to represent the temporal states of web pages and employ dynamic temporal graph embedding to train tamper detection representations for web pages. As for *Q3*, we choose a pretraining approach to optimize node feature representations. Using generative self-supervised learning [7], we perform node feature reconstruction on the dynamic temporal graph, amplifying anomalies within the temporal relation. Based on the solutions to the questions above, we propose a novel web page tampering detection based on dynamic temporal graph pre-training. Combining dynamic temporal graphs with generative self-supervised learning enables the model to capture both the structural evolution and temporal dependencies of web data, while learning representations without labeled samples. This is particularly crucial for detecting subtle, time-dependent tampering patterns that unfold across pages. The method effectively monitors the tampering impact of any Web attack, promptly alerting website administrators to initiate emergency response and security risk investigation, thereby mitigating the loss of online assets in the aftermath of a website system breach. The contributions are as follows:

- We use dynamic temporal graph structure to model the website, named “Website Temporal Graph”. The website temporal graph is a linear arrangement set of graph snapshots in the time series, which is composed of “Website Structure Graph” at different timestamps.
- We introduced the generative self-supervised learning into the dynamic temporal graph for pre-training. Our primary effort lies in adapting the generative self-supervised learning framework to dynamic temporal graphs. We choose DySAT [8] as an autoencoder and decoder to

handle dynamic temporal graph data, and design a temporal masking mechanism to reconstruct node features.

- We optimize the components of the generative self-supervised learning framework to enhance pre-training effectiveness. In DySAT, we incorporate GATv2 [9] to enable it to capture dynamic attention in dynamic temporal graphs. Furthermore, we employ a feedback mechanism combining prediction and reconstruction errors to facilitate autoencoder fitting.
- We propose a method for discovering tampered web pages based on anomaly detection. We construct a temporal graph using content, structural, and statistical features from web pages as node attributes, and perform pretraining to generate node embeddings for downstream anomaly detection. We designed a series of experiments to validate the feasibility of our proposed method, covering aspects such as detection capability, performance, modular design, parameter sensitivity, and algorithmic behavior. Experimental results demonstrate that our approach can effectively identify covert web page tampering in complex dynamic scenarios while maintaining a balanced performance. To support future research, we have made the dataset publicly available [10].

II. RELATED WORK

A. Content Anomaly

Some research focused on detecting anomalies based on web page content, such as detecting malicious links, malicious web pages, or phishing pages. Many malicious and phishing pages result from changes in web page properties due to tampering attacks. Zhiqiang Wu [11] targeted external website links for detection, proposing a malicious website detection method based on multimodal fusion. This method extracted features from external web page texts, dynamic scripts, and images, constructing a deep fusion model that combines these multimodal features to identify anomalous external links. Qian Shao et al. [12] and others suggested using Bert and a ternary decision algorithm to detect hidden links on web pages. They extracted feature vectors encapsulating text information from target external pages and then classified hidden links using a comprehensive decision algorithm. The mainstream research trend for malicious or phishing web page detection involves combining machine learning with web content feature extraction. G Pradeepa et al. [13] introduced various machine learning algorithms for comparison in malicious web page detection, among which random forests achieved the best results. To enhance the training effectiveness of machine learning, MF Rozi et al. [14] aimed to enrich web page features by capturing the intent of web page JavaScript content to improve the abstraction of the webpage’s source code syntax tree. Peng Jiang et al. [15] conducted research on feature engineering for malicious websites from a system provenance analysis perspective, ultimately identifying eight useful features for malicious website detection. Belfedhal et al. [16] proposed a multimodal deep learning framework, processing each type of feature with different deep learning models to enhance the model’s overall analytical capability of web pages. In phishing

page detection, Asadullah Safi et al. [17] found that machine learning techniques were applied in as many as 57 variations within the field. Among them, random forests were the most used, but convolutional neural networks achieved the highest accuracy. Mahdi Bahaghighat et al. [18] evaluated eight major unique scenarios of phishing websites, proving that most machine learning methods could excellently perform phishing site detection tasks. In the latest research on phishing page detection, Furkan Çolhak et al. [19] introduced dedicated multilayer perceptrons and pre-trained models. They designed a novel feature fusion process, inputting the resulting fusion embeddings into a linear classifier.

Detecting web page tampering by identifying malicious content is a widely adopted approach, often relying on models trained on static features such as text, HTML structure, or embedded links. However, these models tend to struggle with covert, low-amplitude tampering attacks, where feature fluctuations are too minor to trigger alarms. Moreover, their dependence on external links introduces temporal instability. Once a referenced external page becomes invalid or changes, detection performance degrades significantly. These limitations suggest that current content-based methods underexploit the sequential and contextual nature of tampering behavior, failing to capture how content evolves over time as a result of cyber confrontation dynamics. We should not overlook the dynamic information resulting from the temporal sequence of web page tampering as a product of cyber confrontation.

B. Tampering Detection

To detect web page tampering in complex website structures and dynamic environments, website log files have become a crucial data source for identifying it. Web server log files are pivotal data sources for detecting web page tampering. Gaurav Varshney et al. [20] proposed a tampering detection approach reliant on domain expressiveness. They harnessed DNS history to calculate hash values for crucial web pages and employed browser verification to forestall malicious content modifications. A drawback of these methods lies in the potential for attackers to manipulate log files, prompting researchers to explore text-based detection mechanisms. Xiangyu Fan et al. [21] introduced a method grounded in text content to identify web page tampering attacks. Their approach involved extracting semantic features from text via the BiGRU model and incorporating CRF learning for sequence state, resulting in feature-rich text representation. Junlin Gao et al. [22] presented text content detection technology using an enhanced Faster-RCNN model for web page tampering detection. They employed a rectangular convolution kernel to accommodate the text's elongated and narrow characteristics and used the Soft-NMS algorithm to minimize missed detection within densely populated regions. As feature engineering advanced, the scope of feature extraction for web page tampering expanded. Hanji Shen et al. [23] introduced a tampering attack detection method predicated on web page change features. Their methodology refined feature engineering encompassing time, amplitude, frequency, location, and content of web page changes, contributing to training the final detection model.

Senhao Wen et al. [24] devised a comprehensive model for analyzing complete web page associations, combining techniques such as topic analysis, visual similarity assessment, website structure analysis, and external resource correlation to detect malicious behaviors, including tampering. Luk Kim et al. [25] proposed a dynamic analysis-based method to uncover vulnerabilities susceptible to web page tampering. Their approach entailed gathering site information, identifying potential JS code elements prone to tampering, and scanning vulnerability through tampering testing, culminating in a vulnerability mining system.

In conclusion, although tampering detection methods based on logs, text content, and structural analysis have advanced significantly in terms of feature representation and data sources, they still underexplore the structural and temporal relationships between web pages, overlooking how changes tend to propagate along specific paths. Most existing approaches treat pages as isolated entities and lack the capacity to model temporal continuity and cross-page dependencies, making them less effective at capturing unexpected, subtle, or progressive changes that deviate from a page's normal evolution. As a result, they often fail to detect tampering behavior that unfolds gradually, spans multiple pages, or remains within the acceptable thresholds of single-page analysis. Furthermore, the limited utilization of deep learning techniques leads to suboptimal representation learning and hinders the detection of context-sensitive tampering. To address these limitations, we propose a dynamic temporal graph-based approach that integrates both structural and temporal signals to enhance tampering detection in complex web environments.

C. Temporal Graph Pre-Training

Recent spatio-temporal pre-training frameworks mainly target sensor-driven urban or neuro signals and assume three conditions: dense and regularly sampled time series at nodes, short-horizon forecasting or imputation objectives, and stable spatial neighborhoods. Representative examples include PreST-Net [26] for traffic forecasting, spatial-temporal decoupled masked pre-training [27] designed for heterogeneous spatio-temporal patches, unified or multi-subgraph urban pre-training and prompting [28] aiming at city-level foundation models with dense streams, importance-masked neuro-decoding pre-training with fine-grained temporal masking [29], and PPT-GNN [30] designed for network security. In our setting, web pages are content-rich nodes and hyperlinks are edges. The graph evolves irregularly and asynchronously with DOM changes, crawl gaps, and missing timestamps. Edges are long-range and bursty through cross-site redirects. The task depends on structural and semantic generation and reconstruction rather than scalar forecasting. These spatio-temporal pipelines require dense per-node time series and forecasting heads that do not supervise the long-range link and content generation we need. Transferring them would add heavy assumptions and still leave our core challenges unresolved, including irregular dynamics, sparse and noisy timestamps, and co-evolution of text and structure.

Generative self-supervised learning directly supervises structure and attribute generation and has recently advanced for

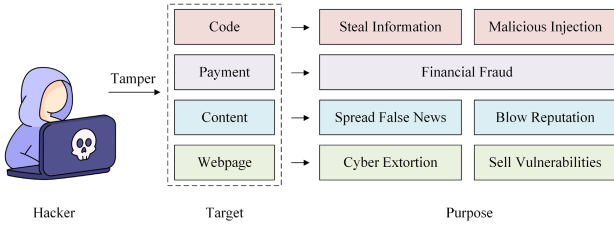


Fig. 1. Hacker's target and purpose for web page tampering attack.

graphs and time series. Dynamic graph masked autoencoding introduces pretexts that reconstruct informative evolution subgraphs instead of predicting a scalar time step, aligning better with topology and content regeneration under irregular clocks as shown in DyGIS [31]. Diffusion-autoregressive hybrids such as TimeDART [32] unify sequence generation with diffusion priors for transferable pre-training under missing or uneven sampling. Graph-generative pre-training such as GraphGPT [33] provides sequence-level graph generation objectives, though with non-trivial tokenization and scalability costs on very large graphs. We therefore build our generative self-supervised learning around DySAT [8] as the backbone. The DySAT with temporal self-attention naturally models asynchronous and bursty link changes, and our pretext tasks such as masked page-attribute reconstruction and future-snapshot edge proposal supervise semantic and structural generation instead of dense forecasting. Newer generative SSL variants that rely on heavy graph tokenization or long diffusion chains are less suitable for million-scale web graphs and are slow to train and infer, while DySAT-based masking and generation remain scalable and directly target the artifacts that define our task, including missing links, volatile content, and cross-page evolution.

III. BACKGROUND

Web page tampering arises from the vested interests of malicious actors. As depicted in Fig. 1, it underscores the primary objectives that drive the initiation of web page tampering attacks. Cybersecurity attackers use web page tampering to covertly access sensitive user information, encompassing login credentials, personal identification particulars, and credit card data. This data extraction might entail minor tweaks to the frontend code of a web page. However, web page tampering may cascade into more profound security repercussions. After web page tampering, attackers may escalate their actions to include additional malicious activities like embedding trojan viruses, hijacking user sessions, and injecting concealed hyperlinks.

Web page tampering can potentially unleash significant financial fraud [34]. Attackers can manipulate data on bank or payment pages, craftily deceiving users into transferring funds to fraudulent accounts. From a societal viewpoint, web page tampering is adept at disseminating deceptive information, enabling the manipulation of public sentiment to exert control over information dissemination. The propagation of false propaganda and the fabrication of news stories could incite social unrest. Within the domain of websites, web page tampering erodes users' confidence in the authenticity of information sources,

potentially catalyzing a crisis of trust between users and website operators. Neglecting to rectify tampering vulnerabilities could profoundly stain a website's reputation. Furthermore, attackers might exploit web page tampering, upload ransomware against website owners, or peddle website vulnerabilities on the dark web, all in pursuit of financial gains.

To differentiate web page tampering from the dynamic changes of web pages, a thorough investigation of the feature of page tampering is essential. According to Trendmicro's analysis report, many of the 1,300,000 website tampering incidents over the past 18 years were acts of expression by hacker organizations with political motives [35]. The objectives pursued by attackers when executing page tampering underscore their intent to sustain the tampered state for an extended duration. Moreover, the adversarial nature of the attacker-defender interaction illuminates various distinctive traits of web page tampering.

- *Concealment:* The concealment of web page tampering mainly falls into two types. Firstly, attackers endeavor to modify page content without leaving traces, making it appear indistinguishable from normal pages. This scenario is often characterized by the embedding of hidden links, where the tampered web page maintains these links for an extended period to contribute to SEO fraud. However, more frequently, attackers opt for visible page tampering to amplify the impact of their modifications to achieve their goals. Thus, the second type of concealment refers to the methods attackers use to alter web pages without triggering tamper detection systems. They exploit covert tampering tactics to bypass defensive measures, enabling unauthorized activities on web pages.
- *Diversity:* The diversity of web page tampering stems from attackers employing various techniques and means to alter content, increasing their chances of success and stealthiness. The most common method involves modifying the web page's source code, including HTML, CSS, and JavaScript. Attackers alter the appearance and behavior of a page by changing HTML tags, stylesheets, or script codes. Replacing images, multimedia, and content on web pages is also a crucial tactic for criminals to spread false information and conduct scams. Moreover, attackers may modify a page's redirection rules or external links to redirect users to malicious sites for information theft or other malicious activities. Aggressively, attackers could directly modify database query statements to manipulate or remove content from databases.
- *Real-time:* Web page tampering can occur in real-time, affecting users as soon as they access the page. Attackers can tamper with the content dynamically based on the specific requests and characteristics of the user, keeping the tampered pages interactive. In actual web sessions, such real-time tampering is more deceptive and concealed. Furthermore, real-time tampering allows attackers to immediately observe the effects of their actions, meaning they can adjust and optimize as needed to ensure the tampered content more effectively achieves their objectives.
- *Customization:* Attackers carry out customized tampering operations on targets based on specific purposes and needs,

making the tampering actions more precise and targeted. This customization is usually goal-oriented. Attackers select specific web pages to tamper with, often because these pages have significant influence, user bases, or sensitive information. They carefully assess the value of targets and develop tampering strategies based on the characteristics of the target website and the behavioral habits of specific user groups. Customized tampering aims to alter content without drawing attention, thus also enhancing the stealthiness of the tampering.

- *Automation:* Under certain conditions, attackers utilize automated tools and techniques for rapid, large-scale tampering operations. For instance, the exposure of some 0-day vulnerabilities can lead to widespread website tampering. Attackers can use tools and scripts to scan numerous web pages and automatically execute code insertion or modification. Automated tools enable attackers to target multiple victims simultaneously, achieving scaled attacks. They can easily manage and control a large volume of tampering actions to maximize their impact.

IV. PRELIMINARY

A. Temporal Anomaly

Contemporary approaches to web page tampering, whether involving scrutinizing the modification time of web page files or comparing data before and after alterations, predominantly rely on anomaly analysis within the framework of temporal data. Detection systems collect and look at old versions of target pages or records of users accessing them. This creates chronological patterns that show normal patterns of behavior. These temporal patterns encompass variations spanning page structure, content, style, and other dimensions. Once a web page undergoes abnormal tampering, deviating from normal behavioral patterns, the detection system will trigger an alert or implement corresponding defensive measures. Consequently, unauthorized file modifications, interface invocations, or changes to web page content inherently manifest as outcomes of anomaly analysis within a temporal context.

The core challenge in time-series AD (Anomaly Detection) lies in precisely modeling the typical behavioral patterns of web pages [36]. The dynamic and highly interactive nature of web pages often engenders frequent alterations in structural and content elements, thus complicating the modeling process. Initial complexities emerge from the characteristics of web page tampering, characterized by concealment and customization. Certain covert and targeted tampering efforts prove difficult to identify as anomalies, sometimes even infusing temporal data to taint detection models. The diversification of web page tampering introduces unpredictability in the types of anomalies, exacerbating the challenge of identifying previously unknown anomalies. Furthermore, the real-time aspect of web page tampering allows for continuous adjustments and refinements, progressively eluding conventional AD techniques.

It's important to note that building detection models based on the typical behaviors of genuine pages significantly increases

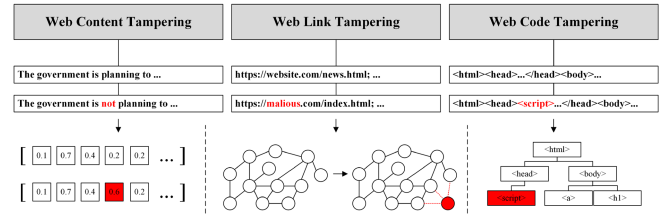


Fig. 2. The graph element anomaly caused by different types of tampering.

the likelihood of false positives. These false alerts place administrators in a perpetual cycle of verification tasked with discerning genuine tampering instances amidst the noise [37]. Consequently, the central thrust of this research endeavor is to effectively utilize time-series data to develop a high-precision web page tampering detection model.

B. Graph Element Anomaly

It is common to convert a website into a graph for analysis [38]. In instances of aberrant alterations within the graph, it is plausible that web pages have been tampered with. Such anomalies predominantly encompass variations in node features and modifications to the website's structure. Fig. 2 shows the corresponding perturbation changes when different elements in the webpage are tampered with. Our capability to detect tampering on web pages is evident when deviations in node features are present. Dynamic page alterations generally manifest as localized text changes. Tampering with textual content by attackers induces drastic semantic shifts that result in perturbations within the node feature vectors. Conversely, inserting malicious code within a web page leads to structural changes in the DOM (Document Object Model) tree [39]. These alterations are absorbed during information aggregation, leading to atypical disruptions in structural features. Concurrently, associated statistical data transforms when attackers tamper with web page elements. This statistical data not only aids in analyzing web pages that have suffered tampering but also facilitates unearthing attackers' primary motives and methodologies in executing the tampering. Structural changes within a website are observed from a broader perspective. Attackers' tampering is mobile, with adjacent pages to the tampered web page susceptible to tampering. Analyzing temporal web page tampering assists in confirming the trajectory and direction of mobile tampering, consequently delving deeper into concealed web page tampering. Additionally, the deletion or addition of specific high-weight nodes within the time series could indicate tampering activities undertaken by attackers.

V. METHOD

A. Framework Overview

We introduce a novel web page tampering detection approach based on dynamic temporal graph pretraining. The overall framework of the method is illustrated in Fig. 3. Initially, the method extracts the essential components for constructing the web structure graph from the website. By capturing snapshots of the graph at different timestamps, a Website Temporal Graph

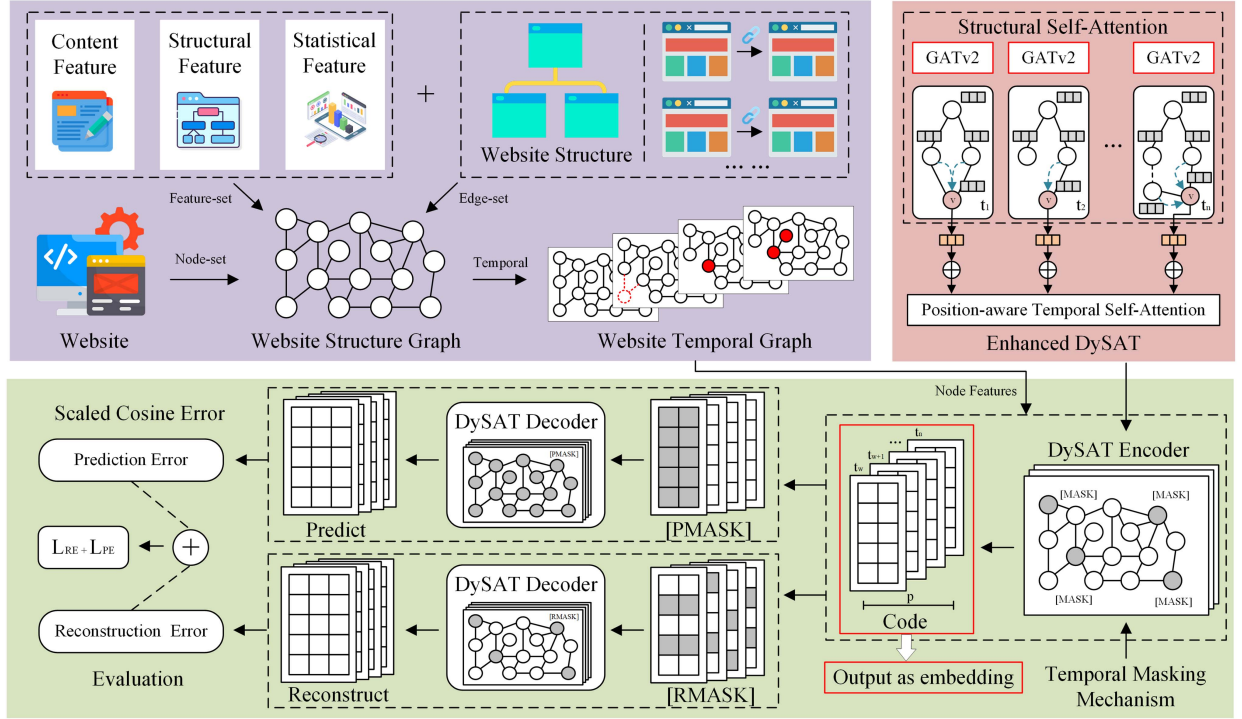


Fig. 3. The framework overview: the purple part represents the construction of the website temporal graph. We extract three types of webpage features as node features, collect edge-set based on the website link structure, and gather multiple timestamped graph snapshots to generate the website temporal graph. The light red part represents the enhanced DySAT model structure. We replace its structural self-attention step in each graph snapshot with GATv2 to capture dynamic attention. The green part represents the pretraining process. We use the enhanced DySAT as the autoencoder and employ a temporal masking mechanism to adapt to temporal data. Additionally, pretraining combines prediction error and reconstruction error to improve encoder fitting.

is generated. Subsequently, modifications are made to the generative self-supervised learning framework to extend its applicability to dynamic temporal graph pretraining. The embedding vectors produced by the pretrained autoencoder serve as inputs to the AD model for identifying tampered pages.

B. Graph Construction

The approach to web page tampering detection involves a comprehensive investigation into the temporal dynamics of website data. Data collection occurs at distinct time intervals, and a structured graph-based framework is meticulously fashioned. The website temporal graph is a robust and purpose-built data structure, capturing the nuanced shifts in website architecture and content over time. This dynamic representation provides a visual narrative of the intricate interplay among pages, links, and various elements across diverse temporal snapshots. This section introduces key information in graph construction:

- Define "Website Temporal Graph" and its basic constituent element "Website Structure Graph";
- Graph node features extraction and edge construction rules;
- Web page tampering analysis theory based on website temporal graph.

Definition 1. *Website Structure Graph [40]:* Within the scholarly context of this research, a core concept is introduced, denoted as the "Website Structure Graph." This graph is constructed within the comprehensive $G = (V, E, X)$ framework, where V signifies the node set and E embodies the edge set.

The feature mapping ensemble is encapsulated within $X = (X_v, X_e)$, wherein X_v encompasses mappings of node features and X_e encompasses mappings of edge features.

To capture the essence of website evolution, a meticulous design and execution of the website structure graph, aligned with the prescribed parameters, is carried out for each distinct moment of website data acquisition. This intricate graph functions as a model, capturing the assimilated website information at a precise juncture and effectively encapsulating its immediate status within the broader temporal panorama. The static aspect of this graph offers an enlightening perspective, unveiling the structural arrangement of web pages and the intricate network of linkages pertinent to a particular snapshot. Using careful comparison and analysis of website structure graphs from various points in time, one can accurately follow the path of the website's evolution and spot oddities like the addition or removal of web pages, changes to the website's basic structure or content, and other noticeable deviations.

Definition 2. *Node and Edge [41]:* Within the network structure graph G , nodes are denoted as $V = \{v_i | 0 < i < (N + 1)\}$, where $N = |V|$. The feature mapping ensemble X_v records the feature vectors corresponding to each node v_i , with the mapping relationship represented by a bijective function $f : V \rightarrow X_v$, where $|V| = |X_v| = N$. Given that the node type remains constant, each feature vector within T_v possesses identical dimensions. Meanwhile, edges are represented as $E = \{e_i | 0 < i < (M + 1)\}$, where $M = |E|$. Each edge connects two nodes, expressed as $e_i = (v_i, v_j)$, with $i, j \in [1, N]$.

Within the website structure graph framework, we have curated three distinct features to serve as the node feature: content features, structural features, and statistical features. Content features encapsulate features that portray the textual information within a web page, incorporating the contextual dependencies and semantic facets of all textual components. Structural features embody attributes delineating the source code structure within a web page. These features are extracted using the DOM tree as a foundational basis to capture the structural essence of the web page. Statistical features encompass attributes that encapsulate statistical data within a web page. These features primarily include statistical information concerning text, images, links, and keywords. We selected 12 statistical features, such as the number of scripts, keyword density, and the number of original tags, that closely relate to tampering behavior.

Each edge within the website structure graph bears directional features, signifying the interlinking relationships between web pages. While constructing the graph structure, we evaluate the involvement of internal web pages and prune low-value leaf node pages. Web pages can exist as isolated islands, devoid of any connections to other pages; such pages are deliberately omitted from our consideration. The greater the number of edges attributed to an internal web page node, the more significant its influence within the website. This heightened influence corresponds to an elevated likelihood of attacker access and tampering.

Definition 3. Website Temporal Graph [42]: We define the temporal graph as $\hat{G} = (G', T')$. Within a period T , time-stamps are systematically recorded at fixed intervals denoted by k , creating the time sequence $T' = \{t_1, t_2, \dots, t_n\}$, where $n = T/k$. The website temporal graph is a linear sequence of website structure graphs along the time series, denoted as $G' = \{G_1, G_2, \dots, G_n\}$. Each graph within the sequence G' is constructed based on website information assimilated during a unitary time interval, substantiating the presence of a mapping function $\varphi : T' \rightarrow G'$. Hence, every temporal node within T' corresponds to a distinct website structure graph.

By meticulously comparing the website structures and links across different temporal points, variations in pages, additions, or deletions can be identified, thereby unveiling potential tampering endeavors. For dynamic websites, the website temporal graph effectively abstracts and analyzes patterns of web page evolution over time, facilitating the identification of concealed tampering assaults by unearthing nodes exhibiting exceptional shifts. Furthermore, the website temporal graph imbues the temporal dimension with significance, affording valuable assistance in pinpointing the temporal junctures of web page tampering incidents. The strength of the website temporal graph lies in its capacity to facilitate the observation of the evolution of graph structures and features along the trajectory of time. Following the training of graph sequences, the change of dynamic web pages often adheres to discernible patterns and predictability. However, when unanticipated changes arise, the web page tampering detection model must swiftly identify these anomalies and undertake recognition. Leveraging the chronicle of website content and structural transformations over time, the website temporal graph can unearth more covert instances of web page tampering. As the content of Web pages undergoes tampering,

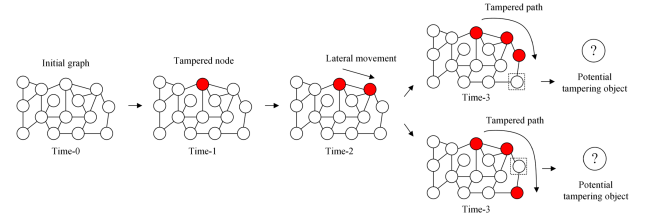


Fig. 4. Tampering path analysis in website temporal graph.

the sequence of page information inevitably shifts, consequently altering the features of page nodes. When compared to the controlled and stable dynamics of regular transformations, tampering causes big changes that cause noticeable changes in the extracted feature vectors.

When malevolent actors implant hidden links or erase connections within a page, the corresponding alterations in the website structure graph ripple through, impacting the construction of target page features during information aggregation. After the completion of specific Web page tampering, it is plausible that the assailant may navigate through links within the page, advancing to the subsequent stage of tampering. Consequently, the likelihood of neighboring tampered pages being influenced is notably elevated. By harnessing this distinctive attribute, we can aggregate information from adjacent nodes of the target page, complemented by analyzing the tampered state of other interconnected pages [43]. Furthermore, we harness the temporal relationships inherent in Web page tampering to dissect the assailant's tampering trajectory, thus unmasking concealed instances of Web page tampering. Fig. 4 shows an example of an attacker's tampering path analysis. Indeed, web pages situated on paths susceptible to tampering are at a heightened risk of being targeted by attackers, warranting greater vigilance and scrutiny. An abrupt, large-scale transformation of the dynamic temporal graph at a given temporal juncture could indicate a profound tampering assault on the website. Through meticulous deep-dive analyses of the website temporal graph, we efficaciously oversee the website's integrity, harnessing the harmonious interplay between graph structures and the rhythmic cadence of temporal mutations to unearth Web pages subjected to tampering.

C. Pre-Training

In web page tampering detection, certain subtle tampering instances are not readily mapped to features. To enhance the classification effectiveness of node features, we propose a dynamic temporal graph pretraining method based on the generative self-supervised learning architecture. The objective of pre-training is to reconstruct node features, enabling the model to learn comprehensive information and thereby improve downstream classification tasks. By using generative self-supervised learning to pre-train website temporal graphs, we can reconstruct node features, thereby enhancing the capture of tampering information. We discuss applying generative self-supervised learning to dynamic temporal graphs by answering the following three questions:

- *Q1*: How to choose autoencoder to process temporal graph?
- *Q2*: How to tune masking strategies based on temporal data?
- *Q3*: How to adjust the reconstruction error in pre-training?

By answering these questions, it can help to understand the flow and reconstruction of temporal data in pre-training.

Q1: Select the DySAT+GATv2 as autoencoder: In generative self-supervised learning, multi-layer GNN (Graph Neural Network) or GAT (Graph Attention Network) are common choices for autoencoders. However, they may not be suitable for handling temporal data structures, which is crucial when dealing with dynamic temporal graphs. The prerequisite for pretraining on dynamic temporal graphs is an autoencoder that can handle temporal data effectively. Given that our input consists of discrete sequences of graph snapshots, methods like TGN (Temporal Graph Network), designed for continuous-time inductive graph learning, are not applicable.

After careful consideration, we opted for DySAT as the autoencoder to take inputs from the website's temporal graph. DySAT aims to capture the structural dynamics of dynamic temporal graphs through a self-attention mechanism, enabling it to learn graph nodes' structural neighborhood and temporal dynamics. DySAT's core components include structural self-attention and temporal self-attention. Structural self-attention aggregates information from neighboring nodes and executes the GAT algorithm on each graph snapshot, followed by weighted summation. However, what sets our approach apart from the original DySAT is the bold replacement of the multi-head attention module in the structural self-attention with a more suitable dynamic attention mechanism. The GATv2 considers the influence of query nodes in the graph structure, optimizing the ranking results of attention weights to achieve dynamic attention. Building on the GAT framework, the GATv2 adjusts the parameters a in the scoring function to yield nonlinear results, and it concatenates the query-key pair before processing. The specific implementation formula is as follows:

$$e(x_i, x_j) = a^\top \text{LeakyReLU}(W \cdot [x_i \| x_j]). \quad (1)$$

The W represents the shared weight matrix, and a denotes a feedforward neural network that maps the concatenated high-dimensional features to the real number space. We replaced the structural self-attention in DySAT with GATv2 to better adapt to the embedding of dynamic temporal graphs. The DySAT's temporal self-attention can capture temporal patterns and aggregate node representations from time steps before t into $v(t)$. After completing the structural self-attention, DySAT performs temporal self-attention to ensure the stability of graph embeddings. To facilitate feasible decoding in subsequent steps, we omit the final aggregation of node embeddings from all timestamps in DySAT. Instead, we retain the encoding outcomes from each individual time slice. As a result, the code vectors produced by the encoder constitute an array of codes across different timestamps, denoted as: $C = \{c(t) | t \in T'\}$. Simultaneously, to match the temporal code input to the encoder, the decoder has also been designed as DySAT to achieve feature reconstruction. This dual

DySAT structure design ensures the stable flow and processing of temporal data within the pretraining framework. This array of codes from DySAT encoder will serve as an intermediate output upon completion of the entire training process, serving as the new node embedding representation. The final vector will be created by averaging the encoding results from all time slices:

$$c_v = \sum_{t=t_w}^{T'} \frac{c_v(t)}{p}. \quad (2)$$

The c_v represents the final node embedding expression for the corresponding node, and w represents the time sliding window length of DySAT. The p represents the number of time slices, calculated from $(n - w + 1)$. In the web page tampering detection task, this vector also represents the feature representation of the web page at the current moment.

The adoption of DySAT+GATv2 as an autoencoder in web-page tampering detection significantly improves the identification and classification of subtle and sophisticated tampering activities by leveraging generative self-supervised learning on dynamic temporal graphs. This method not only captures comprehensive information through the reconstruction of node features but also adeptly handles the temporal dynamics inherent in web structures, making it especially effective in distinguishing between normal updates and malicious alterations.

Q2: Set the temporal masking mechanism: For generative self-supervised learning, masking nodes is a critical step that provides the training direction for feature reconstruction. In conventional graph data, under the condition of maintaining the masking rate, only a subset of nodes needs to be sampled for masking and input into the encoder. However, a new masking strategy needs to be set up for dynamic temporal graphs to apply to node sampling at different timestamps. This new node sampling method is called the "temporal masking mechanism". This mechanism follows the principles of time slices being independent and non-fixed sampling. Under the temporal masking mechanism, node sampling in each time slice is conducted independently, and the masking rate is constrained within the graph of each time slice rather than the entire dynamic temporal graph. It ensures a balanced sampling distribution in the dynamic temporal graph. Furthermore, under the non-fixed sampling rule, nodes sampled in each time slice are not shared, which means that the masking status of the same node may vary at different timestamps. It implies that during self-supervised learning, the autoencoder can refer to node information from previous and subsequent timestamps for encoding, thereby guiding the training direction for feature reconstruction.

Firstly, a subset of nodes $\tilde{V} \subset V'$ is sampled from the dynamic temporal graph, and these nodes are replaced with a masking identifier "[MASK]", denoting their node feature representation as $x_{[M]}$. When inputting a dynamic temporal graph, it is necessary to sample the graph snapshots at different timestamps and set the masking identifier separately. Use the temporal masking mechanism to mask the dynamic temporal graph, the resulting node feature matrix after masking at time t is $\tilde{X}(t) = \{\tilde{x}_i(t) | 0 \leq$

$i < |V(t)|$ }, where:

$$\tilde{x}_i(t) = \begin{cases} x_{[M]}, & v_i(t) \in \tilde{V}(t) \\ x_i(t), & v_i(t) \notin \tilde{V}(t) \end{cases}. \quad (3)$$

Taking into account the intricate structure of the website temporal graph, a masking ratio of 50% is implemented to mitigate redundancy in node features, ensuring the effectiveness of self-supervised learning. The updated dynamic temporal graphs are fed into the DySAT for training upon accumulating substantial amounts of collected data. The temporal encoding obtained by training the DySAT encoder contains multiple time slices. Following the re-masking of latent codes, they are then input into the DySAT decoder for node feature reconstruction. Before inputting the temporal code into decoder, we replace the original masking tag with another, "[RMASK]", which forces the masked nodes to reconstruct their input features from adjacent unmasked latent representations. When implementing the temporal masking mechanism in the decoding process, the re-masked code can be represented as:

$$\tilde{h}_i(t) = \begin{cases} h_{[M]}, & v_i(t) \in \tilde{V}(t) \\ h_i(t), & v_i(t) \notin \tilde{V}(t) \end{cases}. \quad (4)$$

The temporally re-masked code will be input into the DySAT decoder for feature reconstruction. The reconstructed features will be restored to the dimensions of the initial features, and the error of the reconstruction results will be calculated to adjust the coefficients of the autoencoder network.

Implementing the temporal masking mechanism in webpage tampering detection plays a pivotal role by introducing a sophisticated layer of analysis to handle the temporal complexity of web content changes. This mechanism strategically obscures certain node features across different timestamps, compelling the model to infer missing information based on its temporal context and adjacent node attributes. Such a process is crucial for differentiating between normal content evolution and malicious tampering, as it forces the model to recognize patterns of legitimate updates and identify outliers indicative of tampering activities. By simulating scenarios where information is partially hidden, the model becomes more attuned to subtle indications of webpage tampering, enhancing its precision in flagging suspicious alterations.

Q3: Combine reconstruction error with prediction error: In generative self-supervised learning, measuring reconstruction error is a core step in assessing the training state. It calculates the perturbation between the reconstruction and input features, providing feedback to the autoencoder's fitting process. The generative self-supervised learning introduces the scaled cosine error to evaluate the "Reconstruction Error (L_{RE})", which mitigates the impact of feature dimensionality and vector norms. This method adjusts the contributions of different samples in training through a scaling factor, further improving the cosine error. For temporal reconstruction features, scaled cosine error needs to calculate the reconstruction error at each time step and then take the average of the results. The calculation formula is

as follows:

$$L_{RE} = \frac{1}{p} \sum_{t=t_w}^{T'} \frac{1}{\tilde{V}(t)} \left(1 - \frac{(x_i(t))^T z_i(t)}{\|x_i(t)\| \cdot \|z_i(t)\|} \right)^\gamma, \gamma \geq 1. \quad (5)$$

The γ is the scaling factor, and the z denotes the reconstruction output. The p denotes the number of time steps used to compute the average reconstruction error. This term represents the average of masked nodes across all time slices. The scaling factor γ is adaptively adjusted across different datasets, ensuring that different samples are assigned different weights based on the reconstruction error.

In the case of a dynamic temporal graph, relying solely on reconstruction error may not suffice for downstream tasks. It is because the key lies in predicting node information within the current timestamp for web page tampering detection. It implies that data from past timestamps, in the form of graph snapshots, should contribute more to reconstructing node features in the current timestamp. Therefore, we introduce a new metric called "Prediction Error (L_{PE})". This process involves masking all nodes in the current timestamp and subsequently using graph snapshots from the past time window to reconstruct the node features in the same timestamp. The decoder for prediction error is generated by exponentially moving averaging the decoder from the scaled cosine error. Its role is to predict the feature reconstruction for all nodes in the current timestamp. The reconstructed node features for the current timestamp are then input into the scaled cosine error to compute the prediction error. The final error value is obtained by adding the reconstruction error and the prediction error:

$$L = \mu L_{RE} + (1 - \mu) L_{PE}. \quad (6)$$

The variable μ represents the weight in the error calculation, which takes different values in various dynamic temporal graph tasks. The inclusion of prediction error makes self-supervised learning more suitable for dynamic temporal graph pretraining. The autoencoder continuously adjusts its coefficients in response to the calculated error values until it reaches an equilibrium in the training process.

Combining reconstruction error with prediction error in the context of webpage tampering detection introduces a nuanced evaluation metric that significantly amplifies the model's sensitivity to anomalies indicative of tampering. This dual-error approach not only assesses the accuracy of node feature reconstruction but also evaluates the model's ability to predict the current state of a node based on its historical context. The introduction of prediction error is particularly valuable for webpage tampering detection, as it emphasizes the model's capacity to detect deviations from expected temporal patterns, a hallmark of malicious tampering activities. By requiring the model to both accurately reconstruct known node features and effectively predict future states, this method enriches the model's understanding of normal webpage evolution versus tampering-induced anomalies. Consequently, the combination of these errors sharpens the model's ability to discern between benign changes and malicious

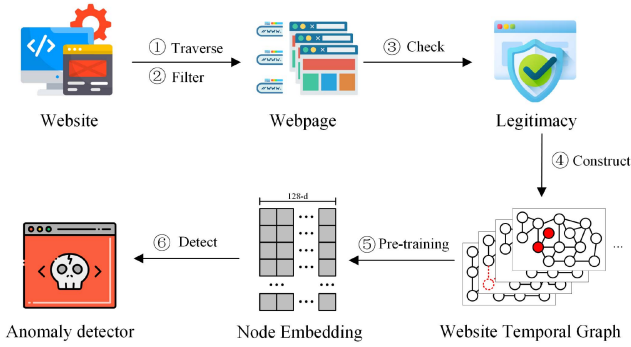


Fig. 5. The tampered web page detection process.

alterations, enhancing the overall robustness and reliability of webpage tampering detection mechanisms.

D. Tampering Detection

Unearthing web page tampering within the dynamic realm of constantly changing web pages presents a formidable challenge. Our objective is centered on harnessing website temporal graphs as a lens to observe whether there are evident disruptions in graph structures and information during instances of tampering. The tampering tends to bring about non-uniform alterations in node features and graph structures, resulting in perturbations that impact the characteristics of specific nodes. These perturbations can be amplified by the pre-training, thereby aiding in identifying whether the intended page has undergone tampering. We propose a web page tampering detection scheme, and its basic process is shown in Fig. 5.

Website administrators, cybersecurity service providers, and relevant government agencies are all potential beneficiaries of this method. A major advantage of this method lies in its consistent approach of collecting website data from a third-party closed box perspective for modeling, eliminating the need for access to the website's source code, directories, or databases. This is particularly important for website operations that prioritize foundational privacy protection. It also means that in practical applications, this method can be directly integrated as a plugin or functional module within conventional Web application firewall software or deployed as a standalone system on servers to monitor multiple websites. This technology routinely gathers relevant website data, automatically updates the website temporal graph, and detects web page tampering activities. The specific steps of the method are as follows:

Step 1. Crawling of web pages from the target Website: We employ a browser-based crawler to collect all web pages from the target website. This crawler is designed to collect the URLs of web pages by strategically positioning hooks at crucial points within the web page's DOM rendering phase. Upon initializing the network crawler, the robot and fuzz dictionaries are integrated into the link pool to expand entry URLs. The crawler automatically performs form population and submission to comprehensively gather exposed entry points to the website, thus complementing intelligent JavaScript event triggering.

Step 2. Filtering of collected web pages: The filtering module encompasses URL deduplication and pseudo-static URL filtration. To enhance the quality of web page collection, we introduce stringent rule mechanisms during the filtering process. The filtering mechanism extracts data from various dimensions such as request method, URL, value, and redirection for URL deduplication, generating MD5 hashes to identify corresponding requests uniquely. Concerning the assessment of static resources, the filtering mechanism creates a mapping type marked with common static resource suffixes. Subsequently, the URL's suffix serves as the key for this map. Finally, thresholds are established based on the number of key parameters and paths, and unique identifiers are allocated to web pages surpassing these thresholds.

Step 3. Security Policy Review for Web pages: We assess the legitimacy of websites through an initial security policy review. These checks include indicators of secure browsing, certificate signatures, resource cross-origin attributes, privacy policies, and more. These tests allow us to confirm that the initial status of the websites under monitoring is not malicious. This effectively prevents the wastage of resources on web page tampering detection and also offers deployment recommendations to administrators.

Step 4. Construction of the website temporal graph: Initially, web page content features are computed using the Bert model. As Bert employs an encoder to handle the semantics, it is adept at capturing long-range dependencies and bidirectional textual information within the web page text. Next, we use Doc2Vec [44] to embed the DOM tree structure of web pages. Each tag on the web page is assigned a unique identifier corresponding to a character. A depth-first traversal strategy is employed to traverse all paths in the DOM tree, arranging the identifiers corresponding to tags in the paths to form words. After all paths are converted, a document composed of words is generated. We train Doc2Vec on these path documents to generate corresponding embedding vectors as structural features. After that, informational and statistical elements within web pages are extracted and normalized into a vector format. These extracted features are included as feature sets in the graph construction process.

All collected web pages within the website are transformed into nodes. Each node's features encompass content, structural, and statistical features. By analyzing the linking relationships on each page, vector edges are established between web pages exhibiting navigational connections. Initiating with the current time as the initial timestamp, the website structure graph is preserved as the initial graph. Subsequently, the following fixed time intervals, corresponding timestamps, and website structure graphs are recorded. The collection of graph snapshots at all timestamps forms the website temporal graph.

Step 5. Pretrain on the dynamic temporal graph: Set the sliding window length to w , and take the graph snapshots from the past w timestamps in the website temporal graph to participate in training. Apply masking to the dynamic temporal graph using a temporal masking mechanism and input it into the DySAT autoencoder for encoding. After re-masking the encoding results, reconstruct the node features through the DySAT decoder. Calculate the reconstruction deviation using a loss function and

fit the autoencoder until the loss reaches the expected level. After complete pretraining, output the encoding results from the last round as new feature vectors.

Step 6. Detect web page tampering using an anomaly detector: We have chosen various state-of-the-art anomaly detectors to test the feature vectors generated during pretraining. These anomaly detectors focus on identifying abnormal components in the input and classifying them as anomalies. These anomaly detectors will collectively determine the identification of web page tampering.

VI. EXPERIMENT

A. Dataset

For data collection, we maintained a list of 600 authorized websites, primarily concentrated in industries such as finance, healthcare, and education. Websites in these sectors have high tampering value due to their data sensitivity and public influence, making them prime targets for hackers. Utilizing concurrency mechanisms, we fully leverage the computational power of multiple CPUs to process and collect nearly 600,000 pages per hour. All 600 websites will be completely crawled within 5 hours, and the data collected will serve as the foundational dataset for the current time slice. Subsequently, the data gathered every 24 hours will be used for the next time slice. This process will be continuously repeated until 21 time slices are generated.

In the course of data collection, we rigorously adhered to the limitations prescribed by the Robots protocol file of each website. Initially, our dataset encompassed over two million web pages sourced from 600 distinct websites. Subsequent filtering was executed based on several criteria: (a) **Page Anomalies:** The task of automatically harvesting data from 600 diverse websites, each developed with different technological frameworks across various sectors, presented substantial challenges. Not all pages were viable; thus, we excluded any page that returned response errors. (b) **Pseudo-static Pages:** Despite efforts to exclude pseudo-static pages during the initial crawling phase, numerous instances were overlooked. Two reviewers independently developed identification rules, resolved interpretation differences through iterative discussion, and jointly applied the unified criteria to manually remove pseudo-static pages. (c) **Homogeneous Technology Stacks:** Throughout the data cleaning phase, we observed that several websites, likely developed by the same entity, shared similar structural attributes. To enhance the diversity of our dataset and manage the data volume effectively, we retained only one representative website from each group of structurally similar entities. (d) **Websites with Insufficient Pages:** Websites that contained fewer than a predefined threshold M , of pages within a single temporal segment were excluded. This criterion was established because a minimal number of pages could lead to an inadequately detailed representation in our dataset analysis.

We also applied security policy review and valid page coverage testing to the web pages. After cleansing, we obtained a dataset comprising over 204,000 clean web pages collected from 75 different websites. We referenced real web page tampering cases from zone-h.org, manually simulating and restoring the

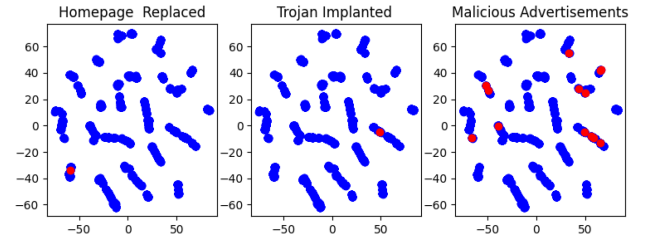


Fig. 6. The t-SNE visualization results of node characteristics of website No. 24.

impact these tampering events had on website pages. To ensure concealment and authenticity, the restoration process involves minimal alterations to links, website content, and source code. Besides, thoroughly taking into account the number of web pages hackers alter in a single attack incident, this standard was then used to proportionally incorporate various tampering categories into our web page dataset, ensuring a realistic representation. We believe that hackers, who alter web page content or insert malicious code, do so to achieve their goals with minimal changes, thus they only tamper with the homepage. In contrast, to increase the click-through rates of ads, malicious advertisers may modify multiple important pages. We consider a webpage significant if the number of links to it surpasses a certain threshold. The methodology for calculating this threshold involves:

$$W_{threshold} = importance_{factor} \times W_{max}. \quad (7)$$

W_{max} is the maximum number of times a page in a website is pointed to, and the $importance_{factor}$ of the paper is set to 0.3. We used t-SNE (T-Distribution Stochastic Neighbour Embedding) to visualize the samples from the 24th website in Fig. 6, demonstrating that positive samples are local anomalies within the entire sample set [45].

After integrating positive and negative web page samples, we created a dataset that records the evolution of web pages over time. Since the negative samples in the dataset comprise malicious tampering activities, we named the dataset the Web-page Tampering Dataset (WPT-Dataset) [10]. We desensitized parts of the dataset that may involve privacy leaks. We employ pseudonymization techniques, using hash functions to convert the original text data into irreversible identifiers, such as names, email addresses, telephone numbers, and addresses. We will not release the original text data of all web pages, but instead publish the extracted feature graphs. The dataset is provided in the form of graph data, where all node attributes have been vectorized. Users will not be able to restore any network identifiers or personal identity information from the dataset. We have made the Dataset publicly available on a data-sharing platform to facilitate similar research endeavors. The composition of the WPT is shown in Table I.

B. Baselines

We selected the SMGCN (Semi-supervised Multi-view Graph Convolutional Networks), DySAT (Dynamic Graph Neural Network), EvolveGCN (Evolving Graph Convolutional Networks)

TABLE I
COMPOSITION OF WPT-DATASET

Type	Snapshot number	Edge	Benign node	Malicious node
Normal web page	1352	3110317	179143	0
Replace homepage	54	169621	7734	54
Implant trojan	57	148535	9475	57
Malvertising inserted	68	165016	7653	1430

for Dynamic Graphs), and GraphMAE (Self-Supervised Masked Graph Autoencoders) on static graphs as baselines for our experiments.

SMGCN [38] achieves state-of-the-art (SOTA) performance in two web page classification datasets through multi-view graph construction and semi-supervised multi-view graph convolutional representation learning. Notably, SMGCN employs semi-supervised learning, and the multi-view convolutional module is implemented using self-supervised learning for adaptation to this scenario.

DySAT [8] employs self-attention mechanisms to aggregate structural neighborhood and temporal dynamics for node representation learning. Comparing our results to DySAT highlights the contributions of our self-supervised learning approach.

EvolveGCN [46] utilizes an RNN (Recurrent Neural Network) to dynamically update internal GNN weights, allowing the model to adapt during testing. Comparing our results to EvolveGCN helps assess our model's ability to evolve.

GraphMAE [7] uses a masked graph autoencoder for self-supervised learning on graphs and has demonstrated superior performance across various tasks on 21 datasets. Comparing our approach to GraphMAE showcases the effectiveness of our dynamic attention aggregation in both the temporal and spatial dimensions within the generative self-supervised framework.

C. Process and Evaluation

We conducted our experiments in the following manner. (a) Pretraining dynamic temporal graph encoders. (b) Node embedding generation. (c) Unsupervised AD model training. (d) Testing on snapshots with positive samples. It's important to note that we didn't employ a single unique AD model but trained 13 different unsupervised AD models, one for each website. This approach allowed us to find each website's most suitable AD model. We believe that different anomaly detectors excel at handling different types of anomalies, and the distribution characteristics of tampered content vary across different websites. Therefore, selecting different AD models tailored to each website can yield better personalized results.

We evaluated our model from five different perspectives:

Q1: How does the model perform on the WPT-Dataset?

Q2: How do hyperparameters affect the model's performance?

Q3: What is the impact of the improved parts of the model on the final results?

Q4: Which AD algorithm should be chosen?

Q5: Is the model's performance and efficiency feasible?

Q6: Is there any real-life example?

For each experiment, we repeated it ten times with different random seeds to ensure result reliability. We used AUC-ROC and AUC-PR as performance metrics, which assess the model's performance on the ROC and PR curves, respectively. All training and testing tasks were implemented using PyTorch 2.0.1+cu118, Python 3.9.17, CogDL, and basic models from PyOD [47]. Besides, we trained the model on a server with two RTX 3090 GPUs.

D. Q1: Comparative Experiment

We adopted AUC-ROC and AUC-PR as the evaluation metrics to assess the model's performance. Table II presents the results for 15 randomly selected websites. We conducted hyperparameter searches using grid search to find the best-performing hyperparameter combinations and then fixed these hyperparameters to repeat the experiment 10 times with different random seeds to compare the statistical results. EvolveGCN achieved the best performance on some websites (e.g., 4 and 73) and showed significantly improved performance compared to using either the spatial-temporal self-attention mechanism or the generative self-supervised framework independently on other websites. We partly attribute this result to EvolveGCN's ability to update the coefficient matrix without relying on node features, making it perform well when dealing with network structures that undergo significant changes in node sets across different snapshots. However, its performance is not consistently stable across different websites and, in most cases, falls short of our model. In the comparative experiments, our model achieved an average AUC-ROC value of 0.9816 and an AUC-PR value of 0.9447. We continue to test all tampered website samples using our model. Fig. 7 displays the specific detection results for all 75 website samples. The AUC values for all websites tampering detection results exceeded 0.8725, with an average value of 0.9602. It indicates that our model exhibits strong adaptability and can effectively detect most cases of webpage tampering.

E. Q2: Hyperparameter Experiment

This experiment evaluated the impact of different encoder/decoder layer depths and attention head numbers on webpage tampering detection performance. We tested nine different hyperparameter combinations, consisting of three different encoder/decoder layer depths (1/2/3) and three different attention head numbers (1/2/4), on 25 randomly selected websites. The experiment results in Table III show that increasing the encoder/decoder layer depth from 1 to 3 leads to a noticeable improvement in both AUC-ROC and AUC-PR scores. The influence of attention head numbers is slightly more complex, with performance varying for different combinations. When using a deeper encoder/decoder architecture, increasing the number of attention heads can significantly enhance AUC-ROC and AUC-PR scores. It suggests a multi-head attention mechanism benefits webpage tampering detection tasks in more complex scenarios.

TABLE II
THE EVALUATION OF 15 RANDOMLY SELECTED WEBSITES UNDER DIFFERENT BASELINES AND OURS

	AUC-ROC(%)														
	1	3	4	9	17	19	42	55	57	58	62	63	65	67	73
SMGCN	74.65± 0.5658	45.09± 0.5608	53.82± 2.1375	58.11± 2.6806	75.75± 0.3201	56.06± 3.0105	51.02± 1.9698	80.64± 0.4766	85.7± 0.4766	63.73± 2.4614	58.65± 1.6858	57.66± 2.4305	71.49± 1.3696	59.58± 2.0405	66.11± 0.0439
DySAT	80.89± 0.115	85.76± 0.0459	67.33± 0.0629	75.54± 0.1804	96.84± 0.0514	76.31± 0.0163	96.58± 0.003	95.26± 0.0111	90.1± 0.1037	79.15± 0.961	83.11± 0.0131	82.16± 0.2111	81.65± 0.1034	86.11± 0.089	93.09± 0.0514
EvolveGCN	97.86± 0.0008	83.4± 0.0402	99.51± 0.0114	98.86± 0.0028	99.12± 0.0004	87.99± 0.1226	85.52± 0.1414	92.37± 0.0028	98.99± 0.0011	97.79± 0.0001	95.19± 0.0032	93.85± 0.0703	95.22± 0.0121	94.26± 0.0321	99.4± 0.0122
GraphMAE	83.89± 0.0802	78.55± 0.0938	68.57± 0.0341	82.81± 0.2423	99.12± 0.0048	81.39± 0.1445	96.12± 0.0041	96.05± 0.1939	90.51± 0.4269	72.84± 0.2414	79.06± 0.4007	81.83± 0.3061	74.86± 0.2919	86.57± 0.0271	82.4± 0.717
Ours	98.11± 0.018	98.68± 0.0159	99.19± 0.006	99.1± 0.0049	100.0± 0.0	98.74± 0.0039	99.44± 0.0021	100.0± 0.0	100.0± 0.0	96.99± 0.0525	98.67± 0.0034	96.28± 0.0053	95.45± 0.0142	93.33± 0.0177	98.41± 0.0091
	AUC-PR(%)														
	1	3	4	9	17	19	42	55	57	58	62	63	65	67	73
SMGCN	12.12± 1.0282	27.46± 1.41	19.13± 2.8774	9.28± 1.5942	51.03± 2.0731	21.45± 3.6963	66.61± 0.8107	32.28± 1.1291	26.37± 0.9656	17.72± 1.0092	21.44± 2.5946	12.38± 4.3744	25.91± 3.2555	15.23± 4.2126	21.09± 1.9043
DySAT	8.65± 0.0066	49.6± 1.8952	24.24± 0.0106	7.02± 0.0064	79.35± 1.1948	26.25± 0.0186	96.4± 0.0094	22.33± 0.1178	10.34± 0.1379	17.82± 0.0262	28.58± 0.0149	20.73± 0.3106	24.92± 0.0789	22.07± 0.1083	44.08± 0.5335
EvolveGCN	39.56± 0.0125	54.61± 0.0799	97.26± 0.022	96.91± 0.0013	74.0± 0.7622	43.54± 0.0793	85.08± 0.0254	14.76± 0.0091	50.0± 0.0722	82.35± 0.0121	62.56± 0.0244	50.16± 1.2654	54.04± 0.0111	66.91± 0.8326	96.36± 0.301
GraphMAE	9.29± 0.0295	36.07± 0.0523	25.59± 0.0086	9.47± 0.1521	86.19± 0.6186	35.32± 1.2493	96.52± 0.0057	55.08± 15.3177	17.82± 2.7422	12.75± 0.0703	27.92± 0.0615	19.84± 0.0359	20.06± 0.0163	20.96± 0.042	29.74± 0.1363
Ours	87.27± 0.2947	94.39± 0.1162	95.81± 0.1991	97.1± 0.0028	100.0± 0.0	92.12± 0.4977	99.78± 0.0002	100.0± 0.0	100.0± 0.0	89.14± 0.1182	97.61± 0.0044	93.89± 0.0058	84.88± 0.0541	89.27± 0.0012	95.8± 0.0451

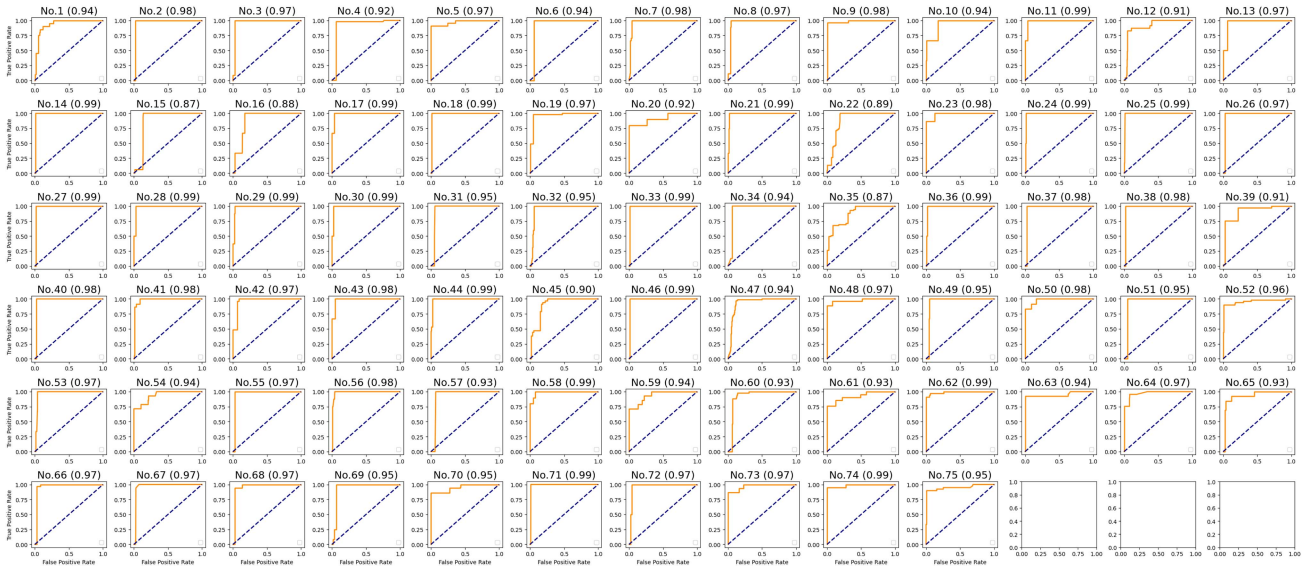


Fig. 7. The ROC curve evaluation results for all websites in WPT-Dataset (AUC value is displayed next to the ID number).

F. Q3: Ablation Experiment

Our approach recommends the joint modeling of spatiotemporal neighborhoods, dynamic attention mechanisms, prediction errors, and a generative self-supervised training framework. To evaluate this design, we conducted experiments by removing each of these four features individually and creating four variants. These experiments were performed on datasets from

25 randomly selected websites using different random seeds. The "w/o DySAT" indicates that our encoder did not undergo pretraining on the dynamic temporal graph; instead, it directly employed an enhanced graph attention network (GATv2) on the time windows. The "w/o GATv2" signifies that our encoder used a standard GAT to aggregate spatiotemporal features. The "w/o prediction" means that we did not utilize temporal prediction

TABLE III
EXPERIMENTAL EFFECTS OF DIFFERENT HYPERPARAMETER COMBINATIONS

Encoder/ Decoder layer	Multi-head at- tention	AUC-ROC(%)	AUC-PR(%)
1	1	94.95±0.3234	72.28±5.5304
	2	94.26±0.4742	74.31±5.8962
	4	94.91±0.4871	74.41±5.8085
2	1	96.16±0.1977	82.9±4.0802
	2	95.66±0.2586	79.43±4.4705
	4	96.32±0.3323	83.75±4.0818
3	1	95.95±0.3511	79.91±4.6216
	2	95.53±0.4783	81.87±4.4739
	4	97.11±0.1337	88.8±3.6099

TABLE IV
ABLATION EXPERIMENTAL RESULTS OF KEY COMPONENTS

Model	Aucroc(%)	Aucpr(%)
Ours	97.58±0.0429	88.25±0.8327
w/o DySAT	91.73±1.1381	67.16±7.7865
w/o GATv2	93.26±0.735	69.88±7.082
w/o prediction	95.99±0.2279	71.22±5.994
w/o MAE	96.35±0.2498	76.61±5.5013

errors and trained solely with masked reconstruction errors. The "w/o MAE" indicates that we did not use a generative self-supervised training mechanism and predicted node attributes directly from data preceding the last time snapshot in the time window.

The experimental results in Table IV demonstrate that the absence of any of these key components significantly reduces the model's detection performance, with detection loss rates exceeding 11%. This degradation is rooted in extreme class imbalance (positives $\approx 0.76\%$), which makes AUC-PR hypersensitive to minor positive-sample fluctuations. Each module uniquely suppresses specific noise types: DySAT's 21-snapshot aggregation mitigates temporal noise; GATv2 counters structural noise; MAE and prediction heads regulate parameter/decision noise. Removing any element propagates sampling noise into metric variance (AUC-PR s.d. surges from ± 0.83 to ≥ 5.5). Consequently, the complete model jointly minimizes both mean error and variance, whereas removing any component degrades accuracy and stability.

G. Q4: Selection of AD Models

In this experiment, we compared the performance of 13 different AD algorithms. We conducted over a thousand experiments and recorded the number of times each AD algorithm achieved the best and second-best performance across all experiments. Finally, we visualized this data in five vertically aligned scatter plots in Fig. 8 to facilitate algorithm performance comparison.

Based on the experimental results, we found that the LOF (Local Outlier Factor) [48] algorithm performed the best, achieving the best performance in 24.2% of all experiments. LOF excels at capturing local outliers, contributing to its top performance. DeepSVDD (Deep Support Vector Data Description) [49] followed as the second-best performer. While it didn't consistently

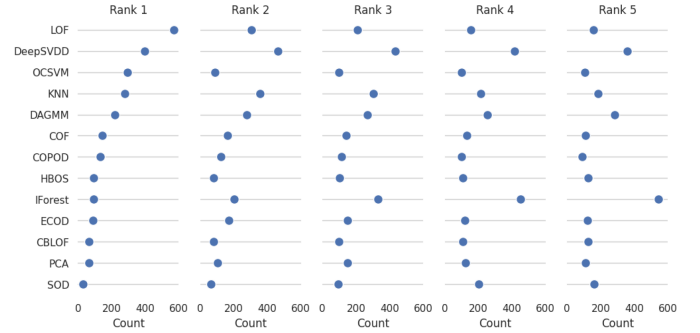


Fig. 8. The performance of AD algorithms.

achieve the best performance like LOF, it demonstrated relative stability with the best performance in 16% of experiments, indicating robustness suitable for specific tampering scenarios. OCSVM (One-Class Support Vector Machine) [50], KNN (K-Nearest Neighbors), and DAGMM (Deep Autoencoding Gaussian Mixture Model) [51] ranked third, fourth, and fifth, respectively, with best performance rates of 11.4%, 11.0%, and 9.2%. Their general applicability allowed them to perform well in a variety of experiments. Isolation Forest [52] may not have excelled in achieving the best performance, but it showed advantages in second-best performance and consistently ranking among the top performers. On the other hand, other presented AD algorithms demonstrated relatively average performance and did not gain significant advantages.

To probe *why* certain detectors excel under specific tampering type, we further evaluate each AD model in a one-vs-normal setting for the three dominant tampering types. Fig. 9 visualises the mean ROC-AUC rank (1 = best) as a 3×13 heat map. A consistent trend emerges: when anomalies are very sparse, as in the *Implant-Trojan* setting, statistics-based SOD attains the lowest rank, whereas denser, locally clustered deviations characteristic of *Malvertising-Inserted* pages are best captured by LOF and, closely behind, DeepSVDD. In contrast, the broad distribution shift caused by a full *Homepage-Replacement* favours distance- and density-aware methods such as COF and SOD. These results emphasise that detector choice should be guided by the underlying distribution of anomalies rather than a single "one-size-fits-all" rule.

H. Q5: Performance Analysis

The Table V provides an overview of the time required for each process in the data preprocessing, model training, and AD phases. Once the model is deployed in a real environment, converting web page text into node features takes only 5 minutes per website. Additionally, the time for anomaly processing presented in the table refers to the total time taken by the 13 different AD models. When our model is actually deployed in a real environment, it will determine a suitable AD model for each website based on the data distribution of its features. It means that the processing time for each website during the AD phase will also be reduced to seconds.

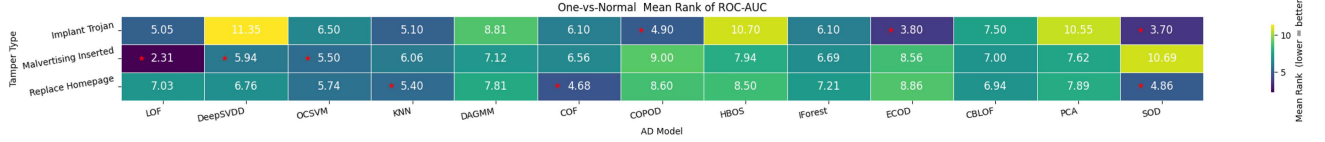


Fig. 9. One-vs-normal mean rank (↓ better) of ROC-AUC for 13 anomaly-detection (AD) models across three web-tampering types. Values denote the average rank among 25 websites; * marks the top-3 models per row.

TABLE V
STAGED MODEL PERFORMANCE TESTING

Process	Time
Parse tag paths	31.5 min per website
Doc2vec	10 min
Extract features	16 min per website
Train an autoencoder	32 min
Dimensionality reduction	7.6 min per website
Graph construction	20 min
Pre-training	≈ 5 min per website
Generate embedding vector	5 sec per website
Abnormal detection	≈ 4 min per website

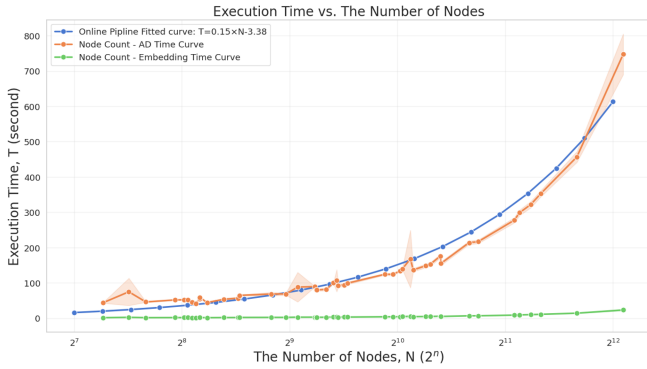


Fig. 10. Empirical scalability of the online stage. Execution time versus the number of nodes (x-axis in 2^n). The orange curve shows total time of running 13 anomaly-detection (AD) algorithms sequentially; the green curve shows embedding-inference time. Shaded regions denote 95% confidence intervals over 10 seeds. The blue line is a fit $T = aN + b$ to the end-to-end online latency (AD + embedding), indicating an approximately linear trend in the sparse-graph regime.

The upstream ETL steps (HTML parsing, Doc2vec, feature extraction) are linear scans over the page corpus and therefore scale linearly with the total number of DOM tags. The main computational cost lies in (i) *DySAT pre-training*, whose time grows as $O(T \cdot E \cdot d)$ with the number of snapshots T , edges E , and hidden dimension d ; and (ii) *anomaly detection*, whose complexity ranges from $O(N \cdot k)$ (COPOD) to $O(N^2)$ (LOF), where N denotes the number of web page nodes in the site.

At deployment time, pretraining is offline; the online latency decomposes as $T_{\text{online}}(N, E) = T_{\text{embed}}(N, E) + T_{\text{AD-total}}(N)$. Here $T_{\text{AD-total}}(N) = \sum_{j=1}^{13} T_j(N)$ aggregates the time of 13 AD algorithms that we evaluate independently and then ensemble at the score level. Embedding accounts for about 5% of the end-to-end time and scales nearly linearly with N on sparse graphs ($E = \Theta(N)$). The orange curve in Fig. 10 reports this

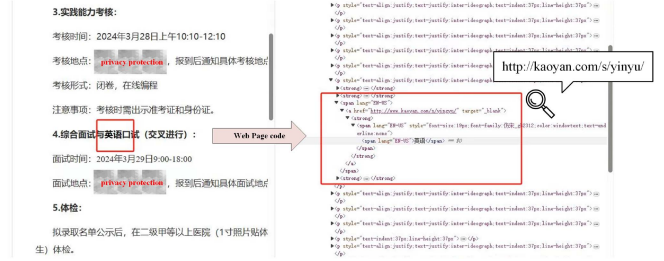


Fig. 11. Real-life cases of web page tampering detection by using our model.

aggregate AD time, the total scaling inherits the order of the dominant detector (i.e., LOF $O(f(N^2))$). In our LOF setting, tree-based k NN back-ends and a small fixed k yield near-linear empirical growth over the evaluated range, which explains why the end-to-end latency (blue) is well approximated by an Ordinary Least Squares fit $T = aN + b$. These results, together with Table V, show that the online pipeline scales gracefully as the site graph grows.

I. Q6: Case Study

With authorization, we deployed a web page tampering monitoring system for a Chinese university and its sub-colleges. The system, hosted on cloud servers as a third-party service, performs real-time data collection and analysis of the university's website. The collection covers all pages under the official website, with data re-collected and website temporal graph updated every 24 hours. On April 1, 2024, the monitoring system issued a tampering alert, pinpointing the official website of a particular college within the university. The college posted a notice for master's degree re-examination on the affected page, as depicted in Fig. 11. The anomaly detector in our model indicated an increase in the probability of page tampering from 2% to 98% within two time intervals. After confirming the authenticity of this web page tampering event, we conducted an in-depth analysis. The re-examination notice page contained an embedded hidden link that pointed to an external website. The hidden link was aimed at a graduate entrance examination service provider, which we suspect may have employed hackers to conduct illegal promotional activities. We have reported the tampering incident to the relevant authorities and completed timely web page repair and vulnerability self-checks. During subsequent continuous monitoring of the same college site, neither our system nor manual reports observed further tampering, suggesting that the remediation and hardening steps reduced the likelihood of similar incidents.

J. Result Analysis

The WPT-Dataset faced significant variations in web page content during the data collection, leading to dynamic changes in the node sets for each snapshot. The results of comparative experiments show that the DySAT and GraphMAE models are not well-suited for web page tampering detection scenarios. We suspect that GraphMAE cannot capture dynamic changes in web pages over time, while DySAT may lack effective guidance from a high-level learning framework. The SMGCN underperforms on our dataset, which we attribute to the lack of key technical expertise in replicating the model, as well as the performance degradation resulting from the transformation of multi-view graph convolution from semi-supervised to self-supervised learning. In contrast, our approach combines DySAT's ability to capture spatiotemporal features with the reconstruction capability of the generative self-supervised framework. It additionally incorporates a dynamic attention mechanism to enhance its feature extraction capabilities and is complemented by a heuristic prediction error, ultimately achieving competitive performance.

In the hyperparameter experiments, the number of encoder/decoder layers has a greater impact on the model's performance. Deeper encoder/decoder structures can better capture data features and anomaly patterns. Choosing a deeper encoder/decoder structure may improve AD performance in practical applications. Furthermore, the final hyperparameter selection should consider the interplay between the number of encoder/decoder layers, attention heads, and dataset complexity. Different datasets and tasks may require different combinations of hyperparameters.

In the ablation experiments, the impact of key components on the model is significant. Among them, the introduction of DySAT has the most pronounced impact, providing a 21% absolute gain in performance. It is because DySAT can aggregate structural neighborhoods and temporal dynamics, allowing it to learn data features from both temporal and spatial dimensions. The dynamic graph attention mechanism, prediction error, and generative autoencoder, respectively, provide gains of 12% to 19%. Such large gains are attributed to the effectiveness of the various model components and the characteristics of the WPT-Dataset, where each website has an average of only 21 snapshots, and the number of nodes in each snapshot is relatively small, leading to larger fluctuations in the experimental results.

The selection and performance analysis of AD models guide the practical application of our model. In the web page tampering detection scenario, the LOF algorithm's strong performance alleviates the difficulty of choosing among diverse AD algorithms. Of course, we can also combine multiple AD algorithms to provide a comprehensive conclusion if slightly lower efficiency is acceptable. The superior performance allows our model to be maintained on servers at a lower cost and complete tampering detection for all web pages within a website in a short time (<10 minutes), replacing the need for global scans and comparisons.

The case study demonstrated that our model can detect web tampering events in real-world college network scenarios, proving its practical value. Besides, our system has been operating

at scale since October 2023, continuously monitoring approximately 600 websites comprising about 150,000 pages, and has identified more than 400 incidents of tampering or abnormal activity across participating organizations. The university deployment described above represents one component of this broader rollout. However, limitations remain, including a 24-hour recrawl interval that may miss short-lived defacements, monitoring restricted to public pages, manual confirmation requirements, and deployment within a single institution. Future work will address these by increasing crawl frequency, expanding deployment to more sites, integrating multi-source telemetry, and conducting red-team tests.

VII. CONCLUSION

In this paper, we propose a web page tamper detection based on dynamic temporal graph pretraining, which improves the effectiveness of web page tamper detection through pretraining on dynamic temporal graphs. The paper provides an overview of web page tampering and discusses web page tamper analysis under temporal and graph. The proposed method is detailed, covering website temporal graph construction, generative self-supervised learning pretraining, and web page tamper detection under anomaly analysis. Experimental results demonstrate that pre-trained node embedding vectors can better distinguish anomalous web pages and perform excellently in various AD classification tasks. The paper also tested the hyperparameter sensitivity and performance of the proposed model, proving the applicability of the proposed model in real-world scenarios.

REFERENCES

- [1] C. Sheng, Y. Yao, W. Li, W. Yang, and Y. Liu, "Unknown attack traffic classification in SCADA network using heuristic clustering technique," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2625–2638, Sep. 2023.
- [2] J. Huo, H. Qu, and L. Liu, "Design and implementation of automatic defensive websites tamper-resistant system," *J. Softw.*, vol. 7, no. 10, pp. 2379–2386, 2012.
- [3] L. Zhao, X. Ling, and F. Deng, "Research on key technology of webpage tampering prevention," *Appl. Mechan. Mater.*, vol. 644, pp. 2755–2758, 2014.
- [4] H. Shuang, L. Zhao, and D. Lie, "vWitness: Certifying web page interactions with computer vision," in *Proc. 53rd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2023, pp. 431–444.
- [5] P. Kanti Kumar, A. Dutta, and P. Kumar, "Application of graph mining algorithms for the analysis of web data," *SSRN Electron. J.*, 2023.
- [6] D. Lewandowski and S. Schultzeiß, "Public awareness and attitudes towards search engine optimization," *Behav. Inf. Technol.*, vol. 42, no. 8, pp. 1025–1044, 2023.
- [7] Z. Hou et al., "GraphMAE: Self-supervised masked graph autoencoders," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 594–604.
- [8] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "DySAT: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 519–527.
- [9] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=F72ximsx7C1>
- [10] wpt-dataset, 2024. Retrieved: Apr. 29, 2024. [Online]. Available: <https://github.com/Polylander/wpt-dataset>
- [11] Z. Wu, "An abnormal external link detection algorithm based on multi-modal fusion," *Int. J. Inf. Secur. Privacy (IJISP)*, vol. 18, no. 1, pp. 1–15, 2024.

- [12] Q. Shao, J. Liu, J. Fang, W. Xiong, and L. Xia, "A novel webpage hiddenlink detection method using bert and ternary decision approach," in *Proc. Int. Conf. Comput. Eng. Netw.*, 2023, pp. 420–431.
- [13] G. Pradeepa and R. Devi, "Malicious webpage detection based on feature fusion using natural language processing and machine learning," in *Proc. 2nd Int. Conf. Edge Comput. Appl. (ICECAA)*, 2023, pp. 486–491.
- [14] M. F. Rozi, S. Ozawa, T. Ban, S. Kim, T. Takahashi, and D. Inoue, "Understanding the influence of AST-JS for improving malicious webpage detection," *Appl. Sci.*, vol. 12, no. 24, 2022, Art. no. 12916.
- [15] P. Jiang et al., "Detecting malicious websites from the perspective of system provenance analysis," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1406–1423, May/Jun. 2024.
- [16] A. E. Belfedhal, "Multi-modal deep learning for effective malicious webpage detection," *Revue d'Intelligence Artificielle*, vol. 37, no. 4, pp. 1005–1013, 2023.
- [17] A. Safi and S. Singh, "A systematic literature review on phishing website detection techniques," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 35, no. 2, pp. 590–611, 2023.
- [18] M. Bahaghighat, M. Ghasemi, and F. Ozen, "A high-accuracy phishing website detection method based on machine learning," *J. Inf. Secur. Appl.*, vol. 77, 2023, Art. no. 103553.
- [19] F. Çolhak, M. İ. Ecevit, B. E. Uçar, R. Creutzburg, and H. Dağ, "Phishing website detection through multi-model analysis of HTML content," in *Proc. Int. Conf. Theor. Appl. Comput., Algorithms Intell. Syst.*, Singapore: Springer Nature, 2025, pp. 171–184, doi: [10.1007/978-981-97-6957-5_15](https://doi.org/10.1007/978-981-97-6957-5_15).
- [20] G. Varshney and N. Shah, "A DNS security policy for timely detection of malicious modification on webpages," in *Proc. 28th Int. Conf. Telecommun.*, 2021, pp. 1–5.
- [21] X. Fan, J. Yang, W. Zhao, J. Deng, and F. Liu, "Webpage tampering detection method based on BiGRU-CRF-RCNN," in *Proc. China Cyber Secur. Annu. Conf.*, Springer Nature, Singapore, 2022, pp. 113–126.
- [22] J. Gao, M. Zhao, J. Wang, Y. Zhao, L. Ma, and P. Yu, "Webpage text detection based on improved faster-RCNN model," in *Proc. Int. Conf. Mach. Learn. Cyber Secur.*, 2022, pp. 54–66.
- [23] H. Shen et al., "Tamper detection based on webpage change features," in *Proc. 23rd Int. Conf. Adv. Commun. Technol. (ICACT)*, 2021, pp. 182–186.
- [24] S. Wen, Z. Zhao, and H. Yan, "Detecting malicious websites in depth through analyzing topics and web-pages," in *Proc. 2nd Int. Conf. Cryptogr. Secur. Privacy*, 2018, pp. 128–133.
- [25] I. L. Kim et al., "Finding client-side business flow tampering vulnerabilities," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 222–233.
- [26] S. Fang, W. Ji, S. Xiang, and W. Hua, "PreSTNet: Pre-trained spatio-temporal network for traffic forecasting," *Inf. Fusion*, vol. 106, 2024, Art. no. 102241.
- [27] M. Zhong, Z. Long, X. Wang, T. Cheng, M. Fang, and L. Chen, "A unified multi-subgraph pre-training framework for spatio-temporal graph," *Knowl.-Based Syst.*, vol. 330, 2025, Art. no. 114428.
- [28] Y. Yuan, J. Ding, J. Feng, D. Jin, and Y. Li, "A universal pre-training and prompting framework for general urban spatio-temporal prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 5, pp. 2212–2225, May 2025.
- [29] Z. Li, Z. Zhu, Q. Li, and X. Wu, "Graph pre-trained framework with spatio-temporal importance masking and fine-grained optimizing for neural decoding," *Pattern Recognit.*, vol. 170, 2025, Art. no. 112006.
- [30] L. V. Langendonck, I. Castell-Uroz, and P. Barlet-Ros, "PPT-GNN: A practical pretrained spatio-temporal graph neural network for network security," in *Proc. 2025 IEEE Eur. Symp. Secur. Privacy Workshops (EuroS&PW)*, 2025, pp. 169–175.
- [31] P. Jiao, X. Zhang, M. Gao, T. Li, and Z. Zhao, "Informative subgraphs aware masked auto-encoder in dynamic graphs," in *Proc. 2024 IEEE Int. Conf. Data Mining (ICDM)*, 2024, pp. 161–170.
- [32] D. Wang, M. Cheng, Z. Liu, and Q. Liu, "TimeDART: A diffusion autoregressive transformer for self-supervised time series representation," in *Proc. 42nd Int. Conf. Mach. Learn.*, PMLR, vol. 267, 2025, pp. 62627–62651.
- [33] Q. Zhao, W. Ren, T. Li, H. Liu, X. He, and X. Xu, "GraphGPT: Generative pre-trained graph eulerian transformer," in *Proc. 42nd Int. Conf. Mach. Learn.*, PMLR, vol. 267, 2025, pp. 77630–77656.
- [34] X. Luo and J. Wang, "Research on internet financial fraud identification based on data mining," *Front. Econ. Manage.*, vol. 3, no. 1, pp. 560–564, 2022.
- [35] trendmicro, "A deep dive into defacement: How geopolitical events trigger web attacks," 2023. Retrieved Mar. 26, 2024. [Online]. Available: https://documents.trendmicro.com/assets/white_papers/wp-a-deep-dive-into-defacement.pdf
- [36] A. Sgueglia, A. D. Sorbo, C. A. Visaggio, and G. Canfora, "A systematic literature review of IoT time series anomaly detection solutions," *Future Gener. Comput. Syst.*, vol. 134, pp. 170–186, 2022.
- [37] M. Albalawi, R. Aloufi, N. Alamrani, N. Albalawi, A. Aljaedi, and A. R. Alharbi, "Website defacement detection and monitoring methods: A review," *Electronics*, vol. 11, no. 21, 2022, Art. no. 3573.
- [38] F. Wu et al., "Semi-supervised multi-view graph convolutional networks with application to webpage classification," *Inf. Sci.*, vol. 591, pp. 142–154, 2022.
- [39] S. Venugopal, S. Y. Panale, M. Agarwal, R. Kashyap, and U. Ananthanagu, "Detection of malicious URLs through an ensemble of machine learning techniques," in *Proc. 2021 IEEE Asia-Pacific Conf. Comput. Sci. Data Eng. (CSDE)*, 2021, pp. 1–6.
- [40] L. Asprino, E. Daga, A. Gangemi, and P. Mulholland, "Knowledge graph construction with a façade: A unified method to access heterogeneous data sources on the web," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–31, 2023.
- [41] L. Gong and Q. Cheng, "Exploiting edge features for graph neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9211–9219.
- [42] T. Erlebach, M. Hoffmann, and F. Kammer, "On temporal graph exploration," *J. Comput. Syst. Sci.*, vol. 119, pp. 1–18, 2021.
- [43] H. Zhang, Y. Chen, X. Li, and X. Zhao, "Simplifying knowledge-aware aggregation for knowledge graph collaborative filtering," in *Proc. Int. Conf. Web Inf. Syst. Appl.*, 2022, pp. 52–63.
- [44] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1188–1196.
- [45] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7499–7519, Jun. 2024.
- [46] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 5363–5370.
- [47] Y. Cen et al., "CogDL: A comprehensive library for graph deep learning," in *Proc. ACM Web Conf.*, 2023, pp. 747–758.
- [48] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. 2000 ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
- [49] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [50] J. Pang, X. Pu, and C. Li, "A hybrid algorithm incorporating vector quantization and one-class support vector machine for industrial anomaly detection," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8786–8796, Dec. 2022.
- [51] W. Khan et al., "DVAEGMM: Dual variational autoencoder with Gaussian mixture model for anomaly detection on attributed networks," *IEEE Access*, vol. 10, pp. 91160–91176, 2022.
- [52] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.



Yijia Xu received the PhD degree in 2024 from Sichuan University, Chengdu, China, where he is currently an associate research fellow in the School of Cyber Science and Engineering. His current research interests include cyber confrontation and web security.



Qiang Zhang received the bachelor's degree in software engineering from Sichuan University, Chengdu, China, in 2018. He is currently working toward the PhD degree in the School of Cyber Science and Engineering with Sichuan University. His current research interests include OS security, intrusion detection, and investigation.



Kaiyang Wang received the Bachelor of Science degree in 2021. He is currently working toward the master's degree in cyberspace security with Sichuan University in Chengdu, China. His current research interests include artificial intelligence security, backdoor attacks, and graph neural networks.



Cheng Huang (Senior Member, IEEE) received the PhD degree from Sichuan University, Chengdu, China, in 2017. From 2014 to 2015, he was a visiting student with the School of Computer Science, University of California, CA, USA. He is currently an associate professor with the School of Cyber Science and Engineering, Sichuan University. His current research interests include web security, attack detection, and artificial intelligence.



Zhonglin Liu received the PhD degree from Sichuan University, Chengdu, China, in 2024. He is currently a full-time teacher with Sichuan University. His current research interests include web security, machine learning, and attack detection.



Yong Fang received the PhD degree from Sichuan University, Chengdu, China, in 2010. He is currently a professor with the School of Cyber Science and Engineering, Sichuan University. His research interests include network security, web security, Internet of Things, Big Data, and artificial intelligence.